

1. Une variable, une case qui porte un nom

Le programme se souvient de quelque chose : c'est tout ce qu'est une variable.

1. Ce qu'il faut comprendre

Une **variable** est une case de la mémoire de la console, à laquelle on donne un nom. `uint8_t vies = 3;` fait trois choses d'un coup : il **réserve** une case, il la **nomme** `vies`, et il y **range** 3.

`uint8_t` est le **type** de la case : un nombre entier, sans signe, sur **8 bits** — un octet. C'est le seul type de la Game Boy : son processeur ne manipule qu'un octet à la fois, et la console n'a que 8 Ko de mémoire de travail pour tout le jeu.

`vies = vies + 2;` se lit **de droite à gauche** : on calcule d'abord `vies + 2` avec la valeur actuelle (3), puis on range le résultat (5) dans la case. Le signe `=` n'est pas une égalité de mathématiques, c'est une **flèche** : « ranger dans ».

`nombre(colonne, ligne, valeur)` écrit la valeur **au moment de l'appel**. Le premier `nombre` a écrit 003 ; changer `vies` ensuite ne le réécrit pas — l'écran ne suit pas la variable tout seul. Dans un jeu, c'est pourquoi on redessine le score après chaque changement.

Les lignes `#include <...>`, **en haut**, disent ce que le programme emploie de la console : `#include <texte>` pour `texte()`, `#include <nombre>` pour `nombre()`. Aucune fonction de la console n'est là d'office : chacune prend de la **place dans la cartouche**, et ne s'emploie que si on l'**inclut**, par son nom écrit comme dans le programme. Sans la ligne, le compilateur refuse, et dit laquelle écrire. Une ligne de trop ne coûte rien.

À toi : ajoute une variable `pieces`, donne-lui 10, retire-lui 4, et affiche-la sur la ligne 8.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **720 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	— la mise en route de la console
main()	59 o	← démarrage	— ta fonction
VBlank	8 o	← la console, à chaque image	— compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	— attend l'image suivante
AttendreVBlank	16 o	← démarrage	— attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre	— attend de pouvoir écrire une case
EcrireTexte	10 o	← texte()	— écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre()	— écrit un nombre en chiffres
CopierTuiles	103 o	← démarrage	— copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	— vide les deux cartes de l'écran
Tuiles	368 o	← CopierTuiles	— 23 lettres sur 43
"VIES:"	5 o	← texte()	— un texte gravé
"APRES LE BONUS:"	15 o	← texte()	— un texte gravé

Lettres gravées : ABEILNOPRSUV0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 2   20 o sur place, dans main() + EcrireNombre (33 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : range un nombre dans une variable, l'affiche,
// le change, puis l'affiche encore.
//
// Tout ce qui suit deux barres « // » est un COMMENTAIRE :
// la console ne le lit pas, il est là seulement pour toi.

#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // main = « principal » : le jeu commence ICI.
                                            // Tout ce qui est entre cette accolade ouvrante
                                            // et la dernière accolade fermante fait partie de main.

    uint8_t vies = 3;                      // Crée une case de mémoire nommée « vies » et y range 3.
                                            // uint8_t = le TYPE de la case : un nombre entier de 0 à 255.
                                            // Le point-virgule « ; » termine chaque instruction,
                                            // comme le point termine une phrase.

    texte(2, 4, "VIES:");                  // Écrit le mot VIES: à l'écran.
                                            // 2 = la colonne (de 0 à 19, de gauche à droite)
                                            // 4 = la ligne   (de 0 à 17, de haut en bas)
                                            // Le texte à écrire se met entre guillemets.

    nombre(8, 4, vies);                    // Écrit la VALEUR de vies (3) en colonne 8, ligne 4.
                                            // nombre() écrit toujours 3 chiffres : on voit 003.

    vies = vies + 2;                       // Se lit de DROITE à GAUCHE :
                                            // 1) on calcule vies + 2, soit 3 + 2 = 5
                                            // 2) on range 5 dans la case vies.
                                            // Le signe « = » veut dire « ranger dans »,
                                            // ce n'est pas le « égal » des mathématiques.

    texte(2, 6, "APRES LE BONUS:");        // Un deuxième texte, deux lignes plus bas (ligne 6).
    nombre(17, 6, vies);                   // vies vaut maintenant 5 : on voit 005.
                                            // Le 003 de la ligne 4 NE change PAS : l'écran garde
                                            // ce qu'on y a écrit, il ne suit pas la variable tout seul.

    while (true) {                         // while (true) = « tant que vrai » : répète pour TOUJOURS
                                            // ce qui est entre les accolades.

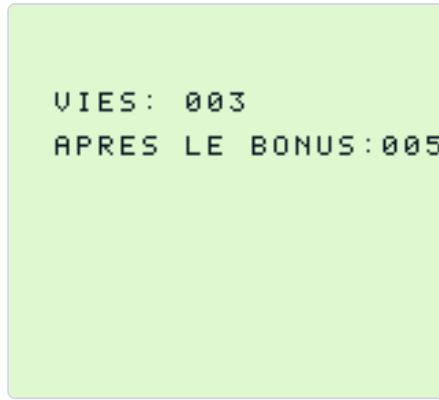
        image();                           // Attend la prochaine image de l'écran (60 par seconde).
    }                                       // Sans cette boucle, main finirait et le jeu s'arrêterait.
}                                          // Fin de main.
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« VIES: 003 », puis « APRES LE BONUS: 005 » deux lignes plus bas.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première valeur écrite est 3
- ✓ après le bonus, l'écran montre 5
- ✓ la case « vies » contient bien 5 (lu : 5)

[← le sommaire](#)

[sommaire](#)

[2. Calculer : + - * / et %](#) →